



1. Промпт как техническое задание для модели

Когда я работаю с AI при программировании, я воспринимаю промпт не как обычный вопрос, а как небольшое техническое задание. Чем точнее я описываю ожидаемое действие, тем меньше модели приходится додумывать самостоятельно.

Например, запрос «улучши этот код» слишком неопределённый. Для одного человека улучшение означает ускорить выполнение, для другого — повысить читаемость, для третьего — исправить архитектуру. Модель сталкивается с той же проблемой и вынуждена сама выбирать направление.

Хороший промпт уменьшает пространство возможных интерпретаций. Я заранее сообщаю, с каким кодом мы работаем, какой результат требуется получить, что разрешено изменять и какие свойства исходной реализации необходимо сохранить.

2. Контекст задачи

Первый важный элемент промпта — контекст. Модели необходимо понимать не только конкретную функцию, но и место этой функции внутри задачи.

Если я просто передам метод `calculate_total()`, модель может предложить формально корректное решение. Но если я уточню, что функция используется в production-интернет-магазине, принимает данные корзины и должна сохранять существующее API, качество решения будет другим.

Контекст особенно важен при работе с существующим проектом. Здесь могут иметь значение версия языка, используемый framework, архитектурный стиль, формат данных и существующие соглашения команды.

Но контекст не должен превращаться в случайное копирование всей документации проекта. Я передаю именно ту информацию, которая влияет на решение текущей задачи.

3. Чётко определяем действие

Следующая часть хорошего промпта — конкретное действие.

Я стараюсь использовать точные глаголы. Если нужно объяснить код, я пишу «объясни». Если требуется изменить реализацию — «отрефакторь». Если нужно только найти дефект — «проанализируй и укажи проблему без изменения кода».

Это особенно важно при работе с агентными инструментами. Фраза «посмотри этот модуль» не сообщает, ожидаю ли я объяснение, review или непосредственное изменение файлов.

Поэтому я стараюсь отделять анализ от модификации. Если сначала нужен анализ, я прямо запрещаю преждевременно переписывать код. Если необходим готовый результат, наоборот, явно указываю, что нужно предоставить изменённую реализацию.

4. Описываем ожидаемый результат

Модели полезно заранее понимать, что именно будет считаться завершённой работой.

Например, при генерации функции я могу указать, что результат должен содержать готовый Python-код, сохранять указанную сигнатуру и корректно обрабатывать определённые входные данные.

При рефакторинге ожидаемый результат другой. Здесь важно сохранить внешнее поведение, убрать конкретную проблему и не менять несвязанные части программы.

Чем точнее определён результат, тем проще проверить ответ. Если после выполнения промпта невозможно понять, соответствует результат требованиям или нет, значит исходная постановка задачи, скорее всего, была недостаточно конкретной.

5. Ограничения защищают от лишних изменений

Ограничения в промпте нужны не для того, чтобы искусственно мешать модели. Они задают границы решения.

Например, я могу запретить добавлять сторонние библиотеки, менять публичный API или перерабатывать архитектуру за пределами текущей задачи. Тогда модель должна искать решение внутри заданных условий.

Это особенно важно при рефакторинге. Без ограничений модель иногда предлагает технически красивый вариант, который требует изменить слишком большую часть проекта. Формально код становится «лучше», но стоимость такого изменения для команды может оказаться неоправданной.

Поэтому хороший промпт сообщает модели не только, что нужно сделать, но и что делать не нужно.

6. Системные инструкции и пользовательский запрос

Теперь важно разделить два уровня инструкций.

Системные инструкции задают постоянные правила поведения модели или агента. Например, они могут определять роль, требования безопасности, используемый стиль разработки или общие ограничения проекта.

Пользовательский промпт задаёт конкретную текущую задачу.

Если я каждый раз повторяю в пользовательском запросе все постоянные правила проекта, промпты становятся перегруженными. Поэтому устойчивые требования лучше хранить на более высоком уровне конфигурации, а в конкретный запрос помещать только информацию, относящуюся к текущей работе.

При этом пользовательский запрос не должен противоречить системным ограничениям. Если системная инструкция запрещает изменение определённых файлов, отдельный промпт не должен автоматически отменять это правило.

7. Few-shot примеры

Few-shot prompting означает, что я не только описываю задачу словами, но и показываю модели несколько примеров желаемого поведения.

Это особенно полезно тогда, когда результат должен соответствовать определённому формату или стилю.

Представим, что мне необходимо генерировать функции в строго определённом стиле проекта. Вместо длинного абстрактного описания я могу показать один или два небольших примера входных данных и правильного результата.

Модель получает дополнительный ориентир и лучше понимает, что я считаю правильным ответом.

Но examples должны быть репрезентативными. Если я покажу пример с ошибочной архитектурой, модель может начать воспроизводить именно эту ошибку. Few-shot не заменяет требования, а дополняет их конкретным образцом.

8. Промпты для генерации кода и для рефакторинга различаются

Генерация нового кода и изменение существующего требуют разных акцентов.

При генерации мне особенно важно описать интерфейс будущего компонента, входные и выходные данные, ошибки и ограничения среды.

При рефакторинге главным становится сохранение существующего контракта. Я должен сообщить, какое поведение нельзя менять, какие тесты уже существуют и какую конкретную проблему необходимо устранить.

Если использовать одинаковый общий промпт для обеих ситуаций, модель может решить задачу слишком свободно.

Поэтому промпт должен отражать жизненный цикл кода. Создание нового модуля допускает больше свободы, чем изменение компонента, от которого уже зависят другие части системы.

9. Промпт для объяснения кода

Отдельный тип задач — объяснение существующей логики.

Здесь я обычно не прошу модель переписывать программу. Мне нужно получить понятное описание того, что уже происходит.

Очень важно определить уровень объяснения. Одному разработчику нужен общий поток данных, другому — подробный разбор сложного алгоритма. Если уровень не указан, модель может либо пересказать код почти построчно, либо дать слишком общее описание.

Хорошее объяснение должно раскрывать причинно-следственные связи. Мне интереснее понять, почему выполняется конкретная проверка и как она влияет на результат, чем просто услышать, что после `if` выполняется следующий блок.

Поэтому в промпте полезно указывать, что нужно объяснить назначение логики, важные условия, изменение данных и возможные ошибки.

10. Промпт нужно оценивать по результату, а не по красоте формулировки

В конце я хочу подчеркнуть, что промпт-инжиниринг не сводится к написанию длинных и сложных запросов.

Хороший промпт — это тот, который стабильно приводит к проверяемому результату с минимальным количеством лишних предположений.

Если короткая инструкция работает устойчиво, нет необходимости превращать её в страницу текста. Если задача сложная и содержит много ограничений, наоборот, слишком короткий запрос может быть недостаточным.

Поэтому я оцениваю промпт практически. Я смотрю, понимает ли модель контекст, выполняет ли именно требуемое действие, соблюдает ли ограничения и можно ли объективно проверить результат.

На следующей практической работе мы будем строить уже не отдельные случайные запросы, а повторно используемые шаблоны промптов для трёх типовых задач разработки: генерации нового кода, безопасного рефакторинга существующей реализации и объяснения сложной программной логики.